REDIS_PROVIDER allows you to perform a manual failover on any hosting platform, even one you manage yourself. You could deploy a second Redis instance and set a new variable named, say NEW_REDIS_URL, to its connection string. Once you change REDIS_PROVIDER to have the value NEW_REDIS_URL and restart your app, your app will be using this new Redis without you having made any code changes. You may never need this, but it's handy to have and minimizes the amount of configuration in your app.

For the rest of the configuration options, you'll use config/sidekiq.yml. Sidekiq parses this as if it were a erb file, so you can put embedded code into it to read from the environment via ENV. You can use this to allow the environment to change the values for concurrency and timeout, falling back to Sidekiq's defaults if the environment variable is not set. There's no need to use this technique for queues or maximum retries, since these values are not likely to require changing without an associated code change. Explicitly setting them to their defaults is a good idea, however, so that it's easier to know where to change them in the future.

snapshots/1-1/sidekiq-book/config/sidekiq.yml

```
:concurrency: <%= ENV.fetch("SIDEKIQ_CONCURRENCY") { 5 } %>
:timeout: <%= ENV.fetch("SIDEKIQ_TIMEOUT_SECONDS") { 25 } %>
:queues:
  - default
```

Configure the Web UI

The last part of the configuration is to set up the Sidekiq Web UI. The Web UI is a Rack App that you can mount onto any path in the app.[6] It's an invaluable resource for understanding and managing Sidekiq in your app (we'll use it in *Handling Permanent Failures via Monitoring* and learn about some of its insights on job behavior in *Sidekiq's Web UI Provides Observability at a Glance*).

First, require sidekiq/web at the top of config/routes.rb like so: